

A HANDS-ON, CODE-FIRST COURSE

LangChain From Zero

Build real GenAI apps with Gemini & local models

PHASE 12 — CAPSTONE PROJECT

Prerequisites: Phases 0–11 — the whole course. This phase assembles all of it.

Targets: a document-Q&A assistant = RAG + tools + agent + memory + tracing + a served API

Estimated time for this phase: 2–4 hours · **Includes:** full glossary · what to learn next · references

The finish line

Every phase has led here. The capstone is one complete application that uses **everything** you've built — and it swaps between cloud (Gemini) and fully-local (Ollama) with a single config flag.

Ph 0 ✓ Environment Setup

Ph 1 ✓ LangChain Fundamentals — Runnables, LCEL, the  pipe.

Ph 2 ✓ Working with Models — invoke/stream/batch, parameters.

Ph 3 ✓ Prompts & Output Parsers — templates, few-shot, structured output.

Ph 4 ✓ Chains & LCEL Composition — parallel, routing, custom Python.

Ph 5 ✓ Memory & Conversational Apps — checkpoint, threads.

Ph 6 ✓ Loading & Splitting Documents

Ph 7 ✓ Embeddings & Vector Stores — FAISS.

Ph 8 ✓ Building a RAG Application

Ph 9 ✓ Tools & Agents — create_agent.

Ph 10 ✓ LangGraph Basics — state, nodes, edges, loops.

Ph 11 ✓ Observability & Deployment — LangSmith, FastAPI.

Ph 12 Capstone Project ← you are here

Assemble it all into one served, traced, memory-enabled, tool-using RAG assistant.

Phase 12 · Capstone Project

WHAT YOU'LL BUILD

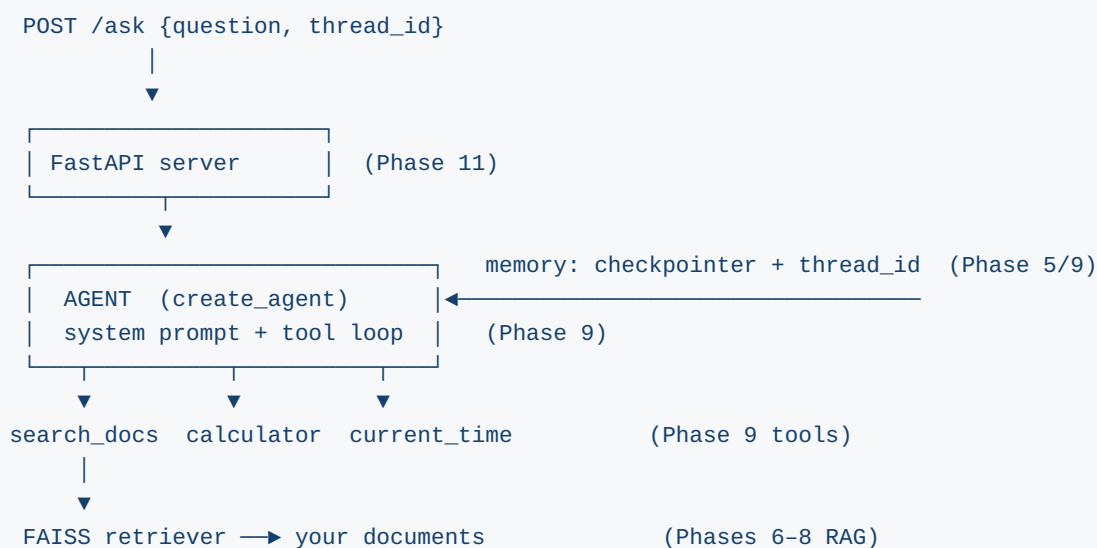
A **Document Q&A Assistant**: a web API where a user asks questions about their own documents. Behind it sits a tool-using **agent** that can search the documents (RAG), do arithmetic, and tell the time; **remembers** each conversation; is **traced** in LangSmith; and runs on **Gemini or Ollama** by flipping one setting.

In this phase

1. 12.1 — The spec & architecture
2. 12.2 — Project layout
3. 12.3 — The swappable backend (one flag)
4. 12.4 — Step 1: Ingest your documents
5. 12.5 — Step 2: Define the tools
6. 12.6 — Step 3: Assemble the agent
7. 12.7 — Step 4: Serve it with FastAPI
8. 12.8 — Running the whole thing
9. 12.9 — Extend it
10. 12.10 — Common errors

12.1 — The spec & architecture

Each piece maps to a phase you've already done — the capstone is integration, not new concepts.



Backend (Gemini OR Ollama) chosen by one flag (Phases 1-2, 7)
Every step traced to LangSmith (Phase 11)

12.2 — Project layout

Five small files, each one responsibility:

```
capstone/
├── .env          # secrets + BACKEND flag + LangSmith vars
├── backend.py    # picks Gemini or Ollama in ONE place
├── ingest.py     # build the FAISS store from your docs (run once)
├── tools.py      # search_docs (RAG) + calculator + current_time
├── assistant.py  # the agent: model + tools + memory
└── server.py     # FastAPI endpoints
```

Your `.env`:

```
BACKEND=gemini          # "gemini" or "ollama"
GOOGLE_API_KEY=Aiza...  # only needed for gemini
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=ls...
LANGSMITH_PROJECT=capstone
```

12.3 — The swappable backend (one flag)

The golden rule from Phase 7 (build & query a store with the *same* embeddings) means the backend choice must drive *both* the chat model and the embeddings — and a separate index folder per backend. Centralise it in `backend.py`:

```
# backend.py
import os
from dotenv import load_dotenv
load_dotenv()

BACKEND = os.getenv("BACKEND", "gemini")      # read the flag
INDEX_DIR = f"faiss_index_{BACKEND}"         # a separate store per backend

def get_chat_model(temperature: float = 0):
    if BACKEND == "ollama":
        from langchain_ollama import ChatOllama
        return ChatOllama(model="llama3.2", temperature=temperature)
    from langchain_google_genai import ChatGoogleGenerativeAI
    return ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=temperature)

def get_embeddings():
    if BACKEND == "ollama":
        from langchain_ollama import OllamaEmbeddings
        return OllamaEmbeddings(model="nomic-embed-text")
```

```
from langchain_google_genai import GoogleGenerativeAIEmbeddings
return GoogleGenerativeAIEmbeddings(model="gemini-embedding-001")
```

WHY A FACTORY FUNCTION

Every other file calls `get_chat_model()` / `get_embeddings()` instead of importing a provider directly. Switching the whole app from cloud to local is now a one-word edit in `.env` (`BACKEND=ollama`) plus re-running ingestion. This is the "swap a line" promise from Phase 1, scaled to a whole app.

12.4 — Step 1: Ingest your documents

Load → split → embed → store (Phases 6–7), saved to disk so the server starts instantly:

```
# ingest.py — run once: python ingest.py mydoc.pdf
import sys
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from backend import get_embeddings, INDEX_DIR

def build(path: str):
    docs = PyPDFLoader(path).load()
    chunks = RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=150).split_documents(docs)
    store = FAISS.from_documents(chunks, get_embeddings())
    store.save_local(INDEX_DIR)
    print(f"Indexed {len(chunks)} chunks into {INDEX_DIR}")

if __name__ == "__main__":
    build(sys.argv[1]) # the PDF path from the command line
```

12.5 — Step 2: Define the tools

Three tools (Phase 9). `search_docs` wraps the retriever and returns chunk text *with sources* so the agent can cite them.

```
# tools.py
from datetime import datetime
from langchain.tools import tool
from langchain_community.vectorstores import FAISS
from backend import get_embeddings, INDEX_DIR

_store = FAISS.load_local(INDEX_DIR, get_embeddings(),
allow_dangerous_deserialization=True)
_retriever = _store.as_retriever(search_kwargs={"k": 4})

@tool
def search_docs(query: str) -> str:
    """Search the user's documents for relevant passages.
    Use for any question about the documents' content."""
    docs = _retriever.invoke(query)
```

```

        return "\n\n".join(
            f"[{d.metadata.get('source', '?')} p.{d.metadata.get('page', '?')}] \n{d.page_content}"
            for d in docs
        )

@tool
def calculator(expression: str) -> str:
    """Evaluate a basic arithmetic expression, e.g. '23 * 17 + 5'."""
    if not set(expression) <= set("0123456789+-*/(). "): # only allow math chars
        return "Error: only numbers and + - * / ( ) are allowed."
    try:
        return str(eval(expression)) # safe-ish: input is restricted above
    except Exception as e:
        return f"Error: {e}"

@tool
def current_time() -> str:
    """Return the current date and time."""
    return datetime.now().isoformat(timespec="seconds")

```

A NOTE ON EVAL

`eval` is dangerous on untrusted input — that's why the calculator first rejects anything but digits and basic math symbols. For a real product, use a proper math parser (e.g. the `mathjs` -style libraries or Python's `ast` -based evaluation) instead of `eval` . Shown here restricted, for brevity.

12.6 — Step 3: Assemble the agent

Now the payoff — one `create_agent` call wires the model, all three tools, a guiding system prompt, and memory (Phases 5 & 9):

```

# assistant.py
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver
from backend import get_chat_model
from tools import search_docs, calculator, current_time

SYSTEM = (
    "You are a helpful assistant with access to the user's documents. "
    "Use search_docs for questions about the documents and cite the source in brackets. "
    "Use calculator for arithmetic and current_time for the date/time. "
    "If the documents don't contain the answer, say so honestly."
)

assistant = create_agent(
    get_chat_model(temperature=0),
    tools=[search_docs, calculator, current_time],
    system_prompt=SYSTEM,
    checkpointer=InMemorySaver(), # memory (swap for SqliteSaver in prod)
)

def ask(question: str, thread_id: str = "default") -> str:

```

```

    config = {"configurable": {"thread_id": thread_id}}
    result = assistant.invoke({"messages": [{"role": "user", "content": question}]},
    config)
    return result["messages"][-1].content

```

12.7 — Step 4: Serve it with FastAPI

```

# server.py
from fastapi import FastAPI
from pydantic import BaseModel
from assistant import ask

app = FastAPI(title="Document Q&A Assistant")

class Query(BaseModel):
    question: str
    thread_id: str = "default"          # same id across turns = remembered conversation

@app.post("/ask")
def ask_endpoint(q: Query):
    return {"answer": ask(q.question, q.thread_id)}

@app.get("/health")
def health():
    return {"status": "ok"}

```

12.8 — Running the whole thing

```

# 0. install the extras if you haven't
pip install -U langchain langgraph langchain-google-genai langchain-ollama \
    langchain-community faiss-cpu pypdf python-dotenv fastapi uvicorn langsmith

# 1. build the index from your document (run once)
python ingest.py handbook.pdf

# 2. start the server
uvicorn server:app --reload

# 3. ask a document question (note the thread_id)
curl -X POST http://localhost:8000/ask -H "Content-Type: application/json" \
    -d '{"question": "What is the refund window?", "thread_id": "sam"}'

# 4. ask a FOLLOW-UP on the same thread — it remembers, then does math
curl -X POST http://localhost:8000/ask -H "Content-Type: application/json" \
    -d '{"question": "Multiply that number of days by 2.", "thread_id": "sam"}'

```

Expected behaviour:

```
{"answer":"The refund window is 30 days from purchase [handbook.pdf p.4]."}
{"answer":"60 days."} # remembered "30" from the previous turn, then used the calculator
```

YOU BUILT A REAL PRODUCT

That second answer is the whole course in one line: it **retrieved** from your document (RAG), **remembered** the earlier turn (memory), **chose and called** the calculator tool (agent), returned over **HTTP** (deployment) — and if you open LangSmith you can **watch every step** (observability). Flip `BACKEND=ollama`, re-run `ingest.py`, and the identical app runs fully offline.

12.9 — Extend it

- **Persistent memory:** swap `InMemorySaver` for `SqliteSaver` so conversations survive restarts (Phase 5 bonus).
- **More tools:** add web search, a weather API, or a database lookup — the agent will use them automatically.
- **Streaming:** add a streaming endpoint so answers appear token-by-token (Phase 2 + 11).
- **Multi-document:** ingest a folder with `DirectoryLoader`; the source metadata already distinguishes them.
- **Better retrieval:** try a larger `k`, re-ranking, or hybrid search (see "What to learn next").
- **A UI:** point a simple React/HTML front-end at `/ask`.

12.10 — Common errors & troubleshooting

Symptom	Likely cause & fix
<code>FileNotFoundError</code> / index not found at startup	You didn't run <code>ingest.py</code> for the current <code>BACKEND</code> , or switched backends without re-ingesting. Re-run ingestion (each backend has its own <code>faiss_index_*</code> folder).
Garbage retrieval after switching backends	Embedding mismatch (Phase 7 golden rule). The <code>INDEX_DIR</code> -per-backend pattern prevents this — just re-ingest after flipping the flag.
Agent doesn't remember across calls	Different (or missing) <code>thread_id</code> . Send the same one each turn.
Ollama: agent ignores tools	Use a tool-calling model (<code>llama3.2</code>); some small models can't call tools (Phase 9).
No traces in LangSmith	<code>LANGSMITH_TRACING</code> /key not set in the server's environment (Phase 11).
Slow first response on Ollama	Model loading into memory; subsequent calls are faster (Phase 0).

CAPSTONE COMPLETE — YOU CAN BUILD WITH LANGCHAIN

You started not knowing what an API key was. You can now build chains, conversational apps with memory, full RAG pipelines, tool-using agents, custom LangGraph workflows, and serve them as traced, production-

shaped APIs — on both cloud and local models. That's the entire core of modern LangChain development. The rest is depth and practice.

Course Glossary

Every key term from Phases 0–12, in one place.

LLM	Large Language Model — takes in text, predicts useful text out.
Token	The chunk an LLM reads/writes (~¾ of an English word); usage is measured in tokens.
Inference	Running a model to get an answer ("calling the model").
API key	Secret string proving you may use a cloud service; sent with each request.
Cloud vs local model	Runs on a provider's servers (Gemini) vs on your own machine (Ollama).
Virtual environment	An isolated per-project Python install for safe package management.
Environment variable / .env	Named values given to programs; <code>.env</code> stores secrets outside your code.
Framework	A reusable code skeleton you build inside; supplies structure and plumbing.
Runnable	LangChain's universal interface: any object supporting <code>invoke</code> / <code>stream</code> / <code>batch</code> .
LCEL	LangChain Expression Language — composing Runnables with the <code> </code> pipe.
Chain	Several Runnables piped together; itself a Runnable.
init_chat_model	Helper that builds a model from a <code>"provider:model"</code> string.
Chat model	Modern model taking a message list, returning an <code>AIMessage</code> ; class starts with <code>Chat</code> .
Message / role	A tagged turn: System (instructions), Human (user), AI (assistant), Tool (tool result).
usage_metadata	Token counts (input/output/total) for a call.
temperature	Randomness dial: low = focused/repeatable, high = creative/varied.
max_tokens	Cap on reply length, in tokens.
stream / batch	Receive an answer in live chunks / run many inputs in parallel.
Prompt template	Reusable prompt with named <code>{blanks}</code> filled from a dict; a Runnable.
ChatPromptTemplate	Builds a list of role-tagged messages from <code>(role, text)</code> tuples.
MessagesPlaceholder	A template slot filled at runtime with a list of messages (e.g. history).
Zero-shot / few-shot	Asking with no examples / including example pairs so the model copies the pattern.
Output parser	A Runnable converting a model reply into a usable Python value.

StrOutputParser	Extracts the plain text string from a model's reply.
Schema / Pydantic	A description of data's shape / the standard library for declaring it as a class.
Structured output	Model output constrained to a schema & validated, via <code>with_structured_output</code> .
RunnableParallel	Runs a dict of Runnables on the same input concurrently; returns a dict.
RunnableLambda	Wraps a plain Python function into a pipe-able Runnable.
RunnablePassthrough / .assign	Forwards input unchanged / adds a computed key to a flowing dict.
RunnableBranch	If/elif/else for chains: <code>(condition, runnable)</code> pairs + a default.
Stateless	The model remembers nothing between calls; it sees only the current request.
Message history	The accumulated turns you resend so the model has context.
Checkpointier	Saves graph state between calls; <code>InMemorySaver</code> (dev), Sqlite/Postgres (prod).
Thread / thread_id	One independent conversation; the key memory is stored under.
Context window	Max tokens a model can read at once; history must fit inside it.
RAG	Retrieve relevant pieces of your data, then have the model answer using them.
Document	LangChain's text container: <code>.page_content</code> + <code>.metadata</code> .
Document loader	Reads a source (file/URL) into Documents (PyPDFLoader, TextLoader, WebBaseLoader).
Chunk / chunking	A small slice of a document / splitting documents into slices.
RecursiveCharacterTextSplitter	Splits on natural boundaries into sized chunks with overlap.
chunk_size / chunk_overlap	Max chunk length / text repeated across boundaries so ideas aren't cut.
Embedding / vector	A list of numbers representing meaning; similar meaning $\Rightarrow$ similar numbers.
Semantic search	Finding text by meaning rather than keywords.
Cosine similarity	A $-1 \dots 1$ score of how aligned two vectors are; higher = more similar.
Vector store / FAISS	A database of chunks + embeddings for fast nearest-vector search / a free local one.
Retriever	A Runnable that returns relevant Documents for a query.
Grounding	Tying answers to provided source text rather than the model's memory.
Hallucination	A confident but false model statement; RAG reduces but doesn't remove it.
Tool / @tool	A function the model may call / the decorator that exposes it to the model.

Agent	A model in a loop that can call tools and reason until it answers.
Tool call / ToolMessage	The model's request to run a tool / the tool's result fed back to the model.
ReAct loop	Reason → Act → observe → repeat; the cycle agents run.
create_agent	The modern API that builds an agent loop from a model + tools (replaces AgentExecutor).
Agentic RAG	Retrieval exposed as a tool, so the agent decides when/whether to search.
LangGraph	Library for workflows as graphs of nodes + edges with shared state; supports loops.
State / node / edge	Shared data (TypedDict) / a step that updates it / a route between steps.
Conditional edge	A runtime branch: a router function returns the next node's name.
Reducer	How a returned field merges into state: overwrite (default) or append (<code>add</code> / <code>add_messages</code>).
START / END	Special markers for a graph's entry and exit (replace <code>set_entry_point</code>).
Observability / LangSmith	Seeing inside your app / LangChain's hosted tracing & monitoring platform.
Trace	A recorded nested timeline of one run: each step's data, latency, tokens, errors.
Deployment / API endpoint	Running your app for others / a URL clients send requests to.
FastAPI / uvicorn	A Python web framework / the server that runs it.
LangGraph Platform	Managed hosting for LangGraph agents/graphs with persistence & scaling.

What to Learn Next

You've covered the core. These are the natural next steps, roughly grouped — none are required to build useful apps, but each deepens a specific area.

Better retrieval (improve RAG quality)

- **Advanced retrievers:** Maximal Marginal Relevance (MMR) for diversity, multi-query retrieval, and the parent-document retriever (search small chunks, return larger context).
- **Hybrid search & re-ranking:** combine semantic + keyword (BM25) search, then re-rank results with a cross-encoder for precision.
- **Production vector DBs:** graduate from FAISS to Chroma, pgvector (Postgres), Pinecone, or Weaviate for scale and metadata filtering.

Stronger agents

- **Middleware:** LangChain's agent middleware for human-in-the-loop approval, automatic summarization, guardrails, PII redaction, and retries (introduced around Phases 5/9).
- **Multi-agent systems & subagents:** coordinating several specialised agents; and **Deep Agents** for batteries-included agents with context compression and a virtual filesystem.
- **Custom LangGraph:** richer graphs — parallel branches, human pause/resume with interrupts, durable execution.

Quality, evaluation & safety

- **Evaluation with LangSmith:** build datasets, run automated evals, and track quality over time instead of eyeballing outputs.
- **Structured output strategies:** tool-calling vs JSON-mode vs provider-native, and when each is most reliable.
- **Security:** prompt-injection defence, tool sandboxing, and output validation — essential once agents take real actions.

Performance & production

- **Async & concurrency:** the `ainvoke` / `astream` / `abatch` methods and async FastAPI for high throughput.
- **Caching & cost control:** prompt/response caching and model routing (cheap model first, escalate when needed).
- **Deployment at scale:** LangGraph Platform, containerisation, and durable DB-backed checkpoints.
- **MCP (Model Context Protocol):** a standard for connecting tools/data sources to models and agents.

HOW TO KEEP LEARNING

LangChain moves fast. The single best habit: when you hit something new, **check the official docs first** (below) rather than older blog posts — this course's biggest lesson is that yesterday's correct code is often today's legacy. Build small projects; each one will pull you into one of the topics above naturally.

Documentation References

The official sources this course is built on. Bookmark these — they're kept current as the libraries evolve.

Primary documentation

- **LangChain (Python) docs:** docs.langchain.com/oss/python/langchain/overview — installation, models, messages, tools, agents, structured output, short-term memory.
- **LangChain API reference:** reference.langchain.com/python — exact signatures for `create_agent`, `runnables`, `prompts`, `integrations`.
- **LangGraph docs:** docs.langchain.com/oss/python/langgraph/quickstart — `StateGraph`, `nodes/edges`, `persistence`, `streaming`.
- **LangSmith docs:** docs.langchain.com/langsmith — `tracing`, `evaluation`, `monitoring`.

Integrations used in this course

- **Gemini (langchain-google-genai):** [chat model & embeddings integration pages](#); get a key at aistudio.google.com.
- **Ollama (langchain-ollama):** [integration page](#); models & install at ollama.com.
- **FAISS vector store:** the `langchain_community.vectorstores.FAISS` integration page.
- **Document loaders & text splitters:** `langchain_community.document_loaders` and the `langchain-text-splitters` package pages.

Deployment & serving

- **FastAPI:** fastapi.tiangolo.com — the web framework used in Phases 11–12.
- **LangGraph Platform:** the deployment section of the LangGraph docs (managed hosting for agents/graphs).

A NOTE ON VERSIONS & DATES

This course targets **LangChain 1.x** (the agent-first generation built on LangGraph). Because the framework changes quickly, a few specifics — model names (e.g. `gemini-2.5-flash`), package versions, and occasionally import paths — may shift after this was written. The *concepts* (Runnables, LCEL, RAG, agents, graphs, tracing) are stable; when a specific call differs from what you see here, trust the official docs above and adapt. Throughout, this course flagged legacy patterns (`LLMChain` , `ConversationChain` , `RunnableWithMessageHistory` , `AgentExecutor` , `initialize_agent` , `LangServe` , `set_entry_point`) so you can recognise and avoid them in older tutorials.

YOU FINISHED LANGCHAIN FROM ZERO

Twelve phases, from "what's an API key?" to a deployed, traced, memory-enabled, tool-using RAG agent that runs on cloud or local models. You didn't just read about LangChain — you typed, ran, broke, and fixed

it. That's how this sticks. Now go build something of your own: the best next phase is a project only you care about. Good luck.